

Moduł 2

Refaktoryzacja do wzorca Value Object

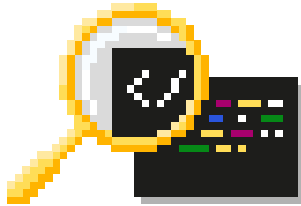
Uszczegółowienie działań refaktoryzacyjnych

Zbudowanie bezpieczeństwa zmiany



Zbudowanie bezpieczeństwa zmiany

1.



Gdzie wprowadzić zmianę?

Wybierz miejsca, gdzie chcesz wprowadzać nowy typ

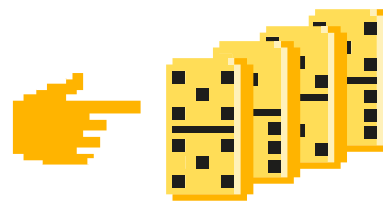
Techniki

- analiza problemów projektowo-systemowych (np. za pomocą event stormingu) podpowiada gdzie szukać (np. problem nowych walut w cenach za przejazd sugeruje szukanie w klasie Transit)
- przeszukiwanie słów kluczowych w kodzie (IDE, ack, synonimy, słowa branżowe)
- przeszukiwanie bazy danych
- wiedza bardziej obytych z systemem programistów i programistek

Na które miejsca się decydujesz? Gdzie zmiany nie wprowadzasz?

- per klasa/moduł/funkcja biznesowa
- zacznij od miejsca, które najbardziej boli (analiza ROI)
- granice serwisów/modułów biznesowych, są dobrym punktem odcięcia zmiany
- analizuj ograniczenia: np. może nie możesz zmienić publicznego API, albo nie masz możliwości dokonywania operacji na bazie

2.



Określenie wpływu zmiany

Jakie są powiązane z wybranym miejscem części systemu? Na co Twoja zmiana będzie miała wpływ? (jakie są obserwowalne zachowania, które mogą zostać dotknięte)

Techniki

- odkrywanie reguł: rozmowa z biznesem - na co wpływa np. poprawność numeru prawa jazdy? Na obserwowalne zachowanie jego aktywacji
- analiza stosu wywołań - Jak dany kod jest wywoływany? Kto to robi i jak?
- analiza zmian w repozytorium (change-coupling) - Jak dane miejsce się zmienia? Co zmienia się razem?
- analiza danych - Jakie zmiany w bazie danych powodują wywołanie danej funkcji? Ile ich jest, skąd pochodzą?

Pułapki

- czasem w zupełnie innej części systemu ktoś uruchamia skrypt bazo-danowy, który polegał na nazwie pola, które zmieniasz - to jest znalezione obserwowalne zachowanie
- często efekty widać zupełnie gdzie indziej w systemie - np. w raportach, widokach lub innych częściach systemu, które korzystają ze zmienianego miejsca bezpośrednio przez bazę danych

3.



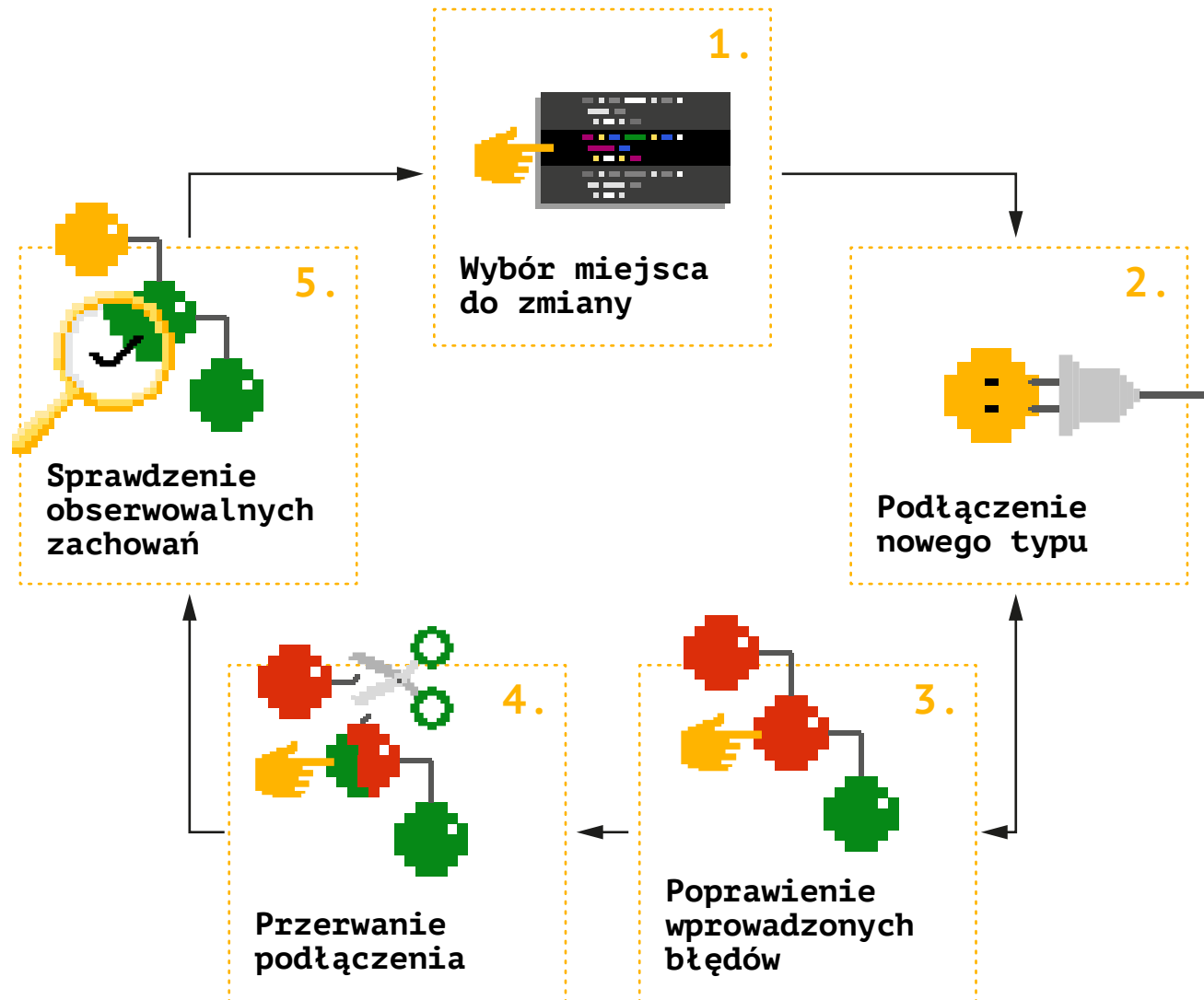
Zapewnienie bezpieczeństwa

Popełnisz błędy. Upewnij się, że w wybranych miejscach przygotowane zostało środowisko, w którym błąd zostanie wyłapany - np. za pomocą testów automatycznych

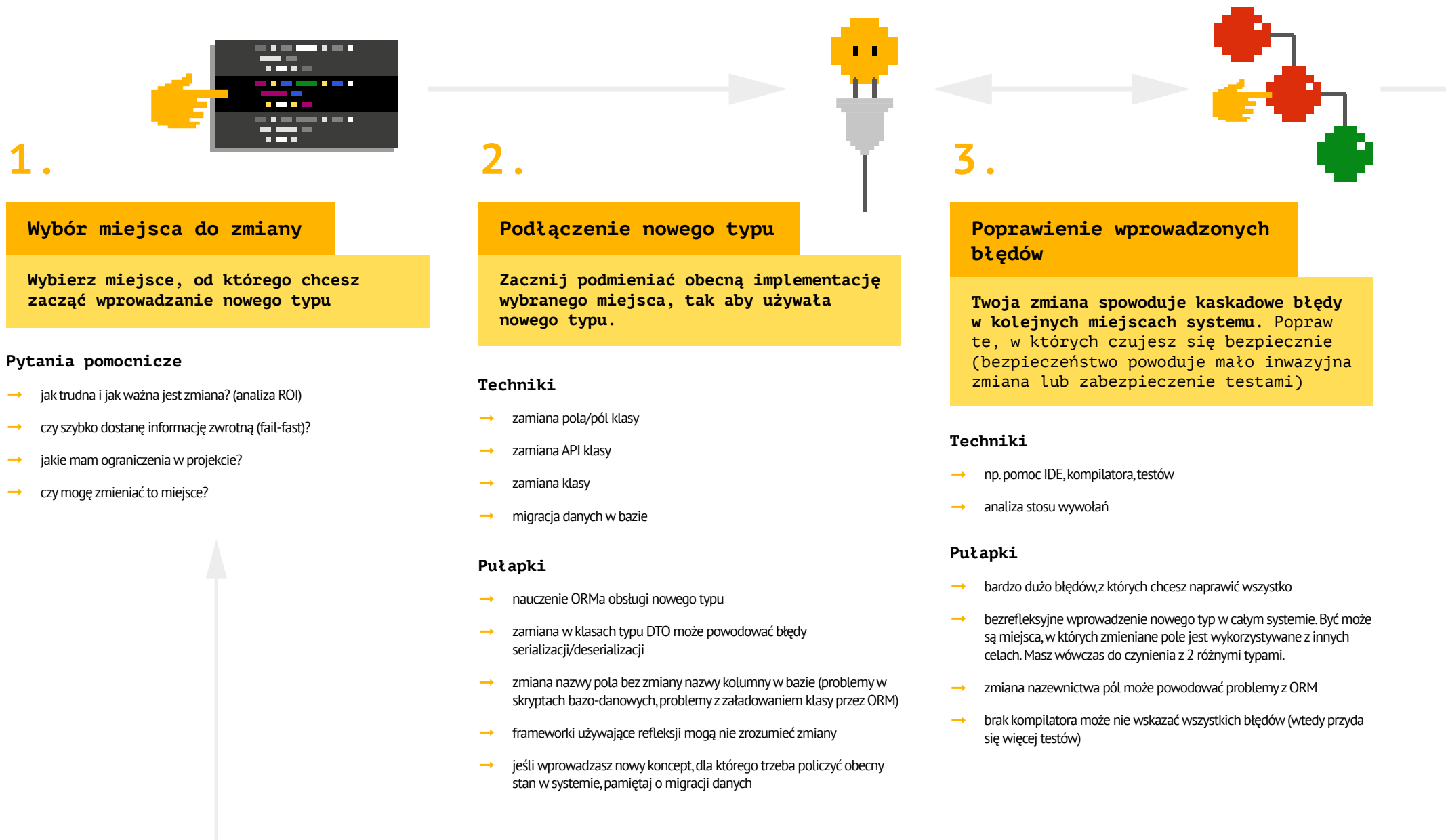
Techniki

- testy automatyczne
- testy charakterystyki
- nie zawsze musisz mieć testy, długa niezrozumiała funkcja, gdzie jedyną dokonywaną zmianą jest zmiana z price na price.toInt() to bezpieczny i mechaniczny refaktoring
- wysokość - testy pisz na wysokości stabilnego interfejsu, który reprezentuje obserwowalne zachowanie.
 - w systemach legacy często oznacza to konieczność wielu testów integracyjnych.
 - jeśli masz logikę w kontrolerach HTTP, test może być nawet testem E2E.
- testy mutacyjne
- testy data-driven

Wprowadzenie zmiany

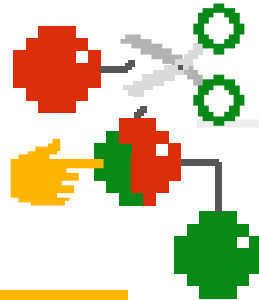


Wprowadzenie zmiany



Wprowadzenie zmiany

4.



Przerwanie podłączania

Nie naprawisz wszystkiego od razu. W bezpieczny sposób odetnij zmiany, które chcesz wykonać dopiero w przyszłości.

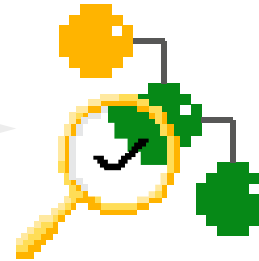
Pytania pomocnicze

- czy odcięcie jest możliwe (czy nowy byt jest "tłumaczalny" do starego?)
- czy w tych kolejnych miejscach czuję bezpieczeństwo?
- czy muszę uzgadniać zmianę z innymi zespołami?
- czy zmiana pociąga za sobą kaskadę innych, czasochłonnych zmian?
- czy wprowadzenie zmiany przyniesie korzyść? (ROI)

Techniki

- metody/projekcje do starych typów
- API odciętego modułu "oszukuje" rozumienie nowego typu za pomocą method overload
- szybka zamiana samego wywołania API danego modułu za pomocą method overload + method inline

5.



Sprawdzenie obserwowalnych zachowań

Czasem popełnisz błąd. Na szczęście system jest na to przygotowany. Wyłap błąd i go popraw za pomocą wcześniej przygotowanych mechanizmów bezpieczeństwa - np. testów automatycznych.

Usuń duplikację z testów. Być może część testów nowego typu wnosi dokładnie to testy zabezpieczające